LoreSpinner Boiler Project — Complete Implementation Plan

0. Executive Decision

Build **Boiler** as a new pipeline layer.

Do **not** solve this only by tuning the runtime prompt to write shorter narration.

Prompt tuning can improve surface pacing, but it cannot solve the deeper issue: runtime is still being handed too much undigested source/event material and is forced to decide live what is playable, what should be skipped, what should become a choice, and what should become an echo.

Boiler solves the structural problem.

Core definition:

> Boiler is the build-time compiler that turns source events into the playable version of a session.

It produces the **Playable Session Cut**: a compact, fun, fast-paced, interactive staging plan that gives runtime a manageable world to perform instead of a long script to narrate.

Current LoreSpinner already has the right foundation: canon remains the source of truth, adaptation controls how canon is experienced, and the narrator executes predesigned session plans rather than inventing structure live. Boiler extends that same principle from “where does the session start?” into “how does the whole session play?”

---

1. Product Goal

Boiler’s goal is to make every LoreSpinner session feel like:

> You are inside the author’s world, driving the story forward.

Not:

> You are reading a narrated version of the script with occasional choices.

Boiler must improve:

1. Pacing.

2. Fun.
3. Player agency.
4. Replayability.
5. Runtime context manageability.
6. Canon fidelity.
7. Author voice preservation.
8. Session-to-session retention.

---

2. Core Philosophy

2.1 Canon remains the source of truth

Events are still the canonical backbone.

Boiler does not rewrite the story into a new plot.

Boiler decides how canon becomes playable.

```
Canon = what happens.
Adaptation = how the session is structured.
Boiler = how the session is played.
Runtime = how the playable scene is performed.
```

This aligns with the current system's mental model that canon is what happens and adaptation is how it is experienced.

2.2 Runtime should not decide the play cut

Runtime should not receive Events 1–20 and decide what matters live.

Runtime should receive:

```
Current playable scene card.
Immediate pressure.
Allowed player affordances.
Canon facts to preserve.
Material to avoid.
Exit condition.
Next scene bridge.
```

2.3 Boiler is not a summary

A summary is dead.

Boiler is a play compiler.

It does not simply compress events.

It classifies them into runtime roles:

```
PLAY
CHOICE
REACTION
MONTAGE
ECHO
CUT
```

2.4 Boiler and Voice Profile are separate

Voice Profile controls how the story sounds.

Boiler controls how the story plays.

Voice Profile gives author DNA, anchor card, and runtime self-check. Boiler must consume those fields, but it must not replace them. V2.3 already established that runtime voice needs concrete anchors, compact rules, and self-check behavior rather than vague instructions.

---

3. Where Boiler Fits in the Pipeline

Current pipeline:

```
IpTrimmingMerge
→ FormatDetection
→ IpAudit
→ VoiceLock
→ StorySessionMap
→ EntryPointDiagnosis / Phase 3
→ SessionArchitecture / Phase 4
→ ChoiceDesign / Phase 5
→ ConsequenceMapping / Phase 6
→ SessionClose / Phase 7
→ EditorialVerification / Phase 8
```

```
→ RuntimeNarratorAssembly
→ Live ChaosEngineService runtime
```

Recommended new pipeline:

```
IpTrimmingMerge
→ FormatDetection
→ IpAudit
→ VoiceLock
→ StorySessionMap
→ EntryPointDiagnosis / Phase 3
→ SessionArchitecture / Phase 4
→ ChoiceDesign / Phase 5
→ ConsequenceMapping / Phase 6
→ SessionClose / Phase 7
→ BoilerGeneration / Phase 7.5
→ EditorialVerification / Phase 8
→ RuntimeNarratorAssembly
→ Live ChaosEngineService runtime
```

Why Phase 7.5?

Boiler needs to know:

- Session map.
- Entry point.
- Beat architecture.
- Designed choices.
- Consequences.
- Session close and hook.
- Voice profile.
- Format type: screenplay or novel.

So it should run after SessionClose.

EditorialVerification should then verify the Boiler output too.

---

4. Database / Storage Plan

4.1 Add fields to `session_adaptations`

Add nullable fields:

```
$table->json('playable_session_cut')->nullable();
$table->string('boiler_status')->default('pending');
$table->string('boiler_version')->nullable();
$table->text('boiler_error')->nullable();
$table->timestamp('boiler_generated_at')->nullable();
```

Recommended statuses:

```
pending
generating
completed
failed
skipped
```

4.2 Optional game runtime fields

Add either explicit columns or store in existing runtime state JSON.

Recommended explicit fields if current game-state handling allows:

```
$table->string('current_playable_scene_id')->nullable();
$table->unsignedInteger('current_playable_scene_index')->nullable();
$table->unsignedInteger('playable_scene_turn_count')->default(0);
$table->json('completed_playable_scene_ids')->nullable();
$table->json('boiler_runtime_log')->nullable();
```

If you do not want new game columns yet, store these in existing game state / metadata JSON:

```
{
  "boiler_runtime": {
    "current_scene_id": "S1_SCENE_01",
    "current_scene_index": 1,
    "scene_turn_count": 2,
    "completed_scene_ids": [],
    "scene_log": []
  }
}
```

4.3 Backward compatibility rule

If a story/session does not have `playable_session_cut`, old runtime behavior can continue.

But if a story is marked `boiler_enabled = true`, missing Boiler output should fail loudly.

No silent fallback for Boiler-enabled stories.

V2.3's philosophy should carry over here: no degraded prompt, no hidden omission, no silent fallback when required build-time data is missing.

---

5. New Files to Add

5.1 Agent

```
app/Ai/Agents/Adaptation/BoilerGenerationAgent.php
```

Responsibilities:

- Define JSON schema.
- Validate required sections.
- Call model with system/user prompt.
- Return structured `playable_session_cut`.

5.2 Job

```
app/Jobs/Adaptation/BoilerGenerationJob.php
```

Responsibilities:

- Load story.
- Load session adaptation.
- Load source events in session range.
- Load previous phase outputs.
- Build prompt context.
- Run BoilerGenerationAgent.
- Validate output.
- Store to `session_adaptations.playable_session_cut`.
- Update `boiler_status`.

5.3 Prompt views

```
resources/views/ai/agents/adaptation/boiler-generation/system-prompt.blade.php
resources/views/ai/agents/adaptation/boiler-generation/prompt.blade.php
```

5.4 Schema / validator

```
app/Ai/Schemas/BoilerPlayableSessionCutSchema.php
app/Ai/Validators/BoilerPlayableSessionCutValidator.php
```

5.5 Prompt context builder

```
app/Ai/Adaptation/BoilerPromptContextBuilder.php
```

Responsibilities:

- Build compact phase context.
- Include only necessary source/event material.
- Select format-specific instructions.
- Include voice slice.
- Include choices/consequences/session close.

5.6 Runtime resolver

```
app/Services/Runtime/PlayableSceneResolver.php
```

Responsibilities:

- Resolve current playable scene from game/session state.
- Advance to next scene when exit condition is satisfied.
- Provide current scene card to runtime.
- Map scene to anchor event ID.
- Preserve old event progression fallback.

---

6. Boiler Input Contract

BoilerGenerationJob should pass the model:

6.1 Story-level inputs

```
story title
detected format
protagonist
genre signals
dramatic question
```

```
world reference summary
character reference summary
voice profile slice
```

FormatDetection already identifies screenplay versus novel and key signals like screenplay formatting, present-tense action, protagonist, and genre.

6.2 Session-level inputs

```
session_number
event_range
chapters_covered
primary_dramatic_question
emotional_register
entry_point_diagnosis
session_architecture
choice_design
consequence_mapping
session_close
```

6.3 Source event inputs

For each event in session range:

```
{
  "event_id": 123,
  "event_position": 12,
  "session_number": 1,
  "title": "Nora exits the lab",
  "objective": "What observably happens",
  "source_excerpt": "Short source excerpt or compressed event text",
  "characters_present": [],
  "location": "",
  "dialogue_samples": [],
  "important_objects": []
}
```

Do not pass unlimited full source if not needed.

Boiler should receive enough source to preserve canon, but not so much that it simply mirrors the full event flow.

6.4 Voice inputs

Use a compact voice slice.

Recommended:

```
anchor_card
master_rule_1_hard_bans
3–5 voice_anchor excerpts max
character dialogue ceilings / speech rules
format-specific screenplay_to_prose or novel_to_play rules
```

Do not pass full Voice Anchor unless needed.

The V2.3 rule says full Voice Anchor is expensive and should not be added globally to every downstream slice. Boiler should use a controlled slice, not the entire voice profile.

---

7. Boiler Output Schema

`playable_session_cut` should be stored as structured JSON.

7.1 Top-level schema

```
{
  "boiler_version": "1.0",
  "session_number": 1,
  "target_playtime_minutes": 25,
  "source_format": "SCREENPLAY",
  "playable_promise": "",
  "session_spine": [],
  "event_role_map": [],
  "playable_scenes": [],
  "montage_bridges": [],
  "cut_material": [],
  "must_reintroduce_later": [],
  "session_end_card_spec": {},
  "runtime_pacing_contract": {},
  "verification_notes": {}
}
```

---

8. Event Role Map

Every source event in the session range must receive exactly one role.

```
{
  "event_id": 123,
  "event_position": 7,
  "role": "PLAY",
  "why": "The protagonist is embodied, pressure is live, and the player can
act.",
  "included_in_scene_id": "S1_SCENE_03",
  "canon_must_keep": [],
  "cut_or_compression_note": "",
  "reintroduction_route": ""
}
```

Allowed roles:

8.1 PLAY

Use when the event deserves live staging.

Criteria:

```
protagonist present
immediate pressure
player can act
scene can support 2–4 turns
strong sensory/object/NPC hook
```

8.2 CHOICE

Use when the event contains a durable branch decision.

Criteria:

```
clear values in tension
choice affects branch dimension
consequence mapping exists
worth remembering later
```

8.3 REACTION

Use when the event should appear as consequence, not full play.

Example:

```
NPC tone changes.
Door seals.
HUD warning appears.
Object warms.
Ally withholds truth.
```

8.4 MONTAGE

Use when the event matters for movement, atmosphere, or compression, but not interaction.

Example:

```
crossing corridors
travel
repeated threat pressure
procedural setup
visual transition
```

8.5 ECHO

Use when the event is better delayed into a later memory, object, line, or consequence.

Example:

```
cut exposition returns as HUD text
skipped relationship beat returns as NPC reaction
background lore returns as object description
```

8.6 CUT

Use when the event is nonessential to the playable experience and does not need active reintroduction.

---

9. Playable Scene Card Schema

Each playable scene card should look like this:

```
{
  "scene_id": "S1_SCENE_01",
  "scene_order": 1,
  "scene_type": "cold_open_play",
  "source_event_ids": [101, 102],
  "anchor_event_id": 101,
  "title": "The RENDAN Shard",
  "duration_budget": {
    "target_turns": "2-3",
    "max_turns_before_gravity": 3,
    "first_input_max_words": 180,
    "default_response_words": "80-180",
    "setpiece_response_words": "220-320"
  },
  "opening_image": "",
  "live_pressure": "",
  "player_affordances": [],
  "interaction_type": "open_input",
  "choice_links": [],
  "canon_must_keep": [],
  "cool_extracts": [],
  "voice_surface_notes": [],
  "npc_pressure": [],
  "object_anchors": [],
  "world_state_hooks": [],
  "branch_dimension_hooks": [],
  "consequence_hooks": [],
  "forbidden_expansions": [],
  "exit_condition": "",
  "next_scene_bridge": ""
}
```

---

10. Interaction Types

Boiler should not force every interaction into three choices.

Allowed interaction types:

```
open_input
two_option_crisis
three_option_branch
inspection_prompt
dialogue_prompt
body_action_prompt
continue_prompt
fast_forward_prompt
session_end_choice
```

10.1 `open_input`

Use when the scene wants player expression.

Example:

```
What do you do with the shard?
```

10.2 `two_option_crisis`

Use when the world has collapsed to a binary.

Example:

```
Seal the channel.
Open it.
```

10.3 `three_option_branch`

Use when three meaningful values exist.

Example:

```
Defy.
Probe.
Comply tactically.
```

10.4 `inspection_prompt`

Use when the fun is touching/looking.

Example:

```
The pod. The monitor. The shard. Ilya’s hand.
What do you inspect first?
```

10.5 `dialogue_prompt`

Use when the key agency is speech.

Example:

```
Ilya waits.
What do you say?
```

10.6 `fast_forward_prompt`

Use when a scene has no more meaningful live agency.

Example:

```
You can leave now, or linger one beat longer.
```

---

11. Runtime Pacing Contract

Every Boiler output must include:

```
{
  "first_input_max_words": 180,
  "default_response_words": "80-180",
  "major_reveal_words": "220-320",
  "max_exploratory_beats_before_gravity": 3,
  "max_scene_turns_without_state_change": 4,
  "choice_density_target": "one meaningful player input every 1-3 minutes",
  "montage_allowed": true,
  "do_not_walk_through_cut_events": true
}
```

This updates the current “2–4 exploratory beats then narrative gravity” pacing rule into a concrete scene-level budget. V2.3 already removed a hard literal timer and preserved the intent through exploratory beats followed by narrative gravity.

---

12. Format-Specific Boiler Behavior

12.1 Screenplay Boiler

Screenplay Boiler is a **shot-to-play compiler**.

It should extract:

```
image anchors
scene pressure
hard cuts
dialogue confrontations
object beats
visible body actions
cinematic entrances/exits
```

It should not over-novelize.

It should not preserve scene headings as output.

It should treat screenplay action as visual staging material.

The older runtime already understood that screenplay actions are canonical facts but not wording, and that scene headers should not be output directly. Boiler should push this earlier into build-time design.

12.2 Novel Boiler

Novel Boiler is an **interiority-to-play compiler**.

It should extract:

```
desire
fear
secret
contradiction
object of fixation
relationship pressure
```

```
decision moment
sensory anchor
```

It should convert internal pages into external playable pressure.

Example:

```
Three pages of internal doubt
→ one room
→ one object
→ one person waiting
→ one trust choice
```

12.3 Shared rule

Both formats must output the same schema.

Only the prompt strategy differs.

---

13. Boiler Prompt System Rules

The Boiler system prompt should say:

```
You are the Boiler Compiler for LoreSpinner.

Your job is not to summarize the source.
Your job is to turn the session into a fast, playable experience.

The source events are canon.
Do not contradict canon.
Do not invent new major plot.
Do not preserve every event equally.

Classify every event as:
PLAY, CHOICE, REACTION, MONTAGE, ECHO, or CUT.

Prioritize:
- protagonist embodiment
- immediate pressure
- object interaction
- NPC friction
- replayable choice identity
```

```
- consequence visibility
- fast input
- author voice surface

Output only valid JSON matching the schema.
```

---

14. Runtime Integration

14.1 Runtime prompt should support two modes

Old mode

Used when no Boiler exists.

```
current event mode
```

Boiler mode

Used when `playable_session_cut` exists and `boiler_status = completed`.

```
playable scene mode
```

14.2 Boiler mode runtime context

Runtime should receive:

```
session spine compact
current playable scene card
current scene turn count
relevant player history
relevant world state
attached choice/consequence data
voice anchor/card/self-check
```

Runtime should not receive the full session event range as the main active material.

14.3 Current event still exists

`current_event_id` should remain for canonical anchoring.

The current system uses `game.current_event_id → event.session_number → session_adaptations`. Boiler should not break that.

Each playable scene has an `anchor_event_id`.

When scene changes, update:

```
current_playable_scene_id
current_playable_scene_index
playable_scene_turn_count
current_event_id = next_scene.anchor_event_id
```

14.4 Session start

Session start still uses `entry_point_diagnosis.start_event_id`.

This must remain the runtime source of truth.

Current Entry Point already stores `start_event_id`, `cold_open`, `emotional_promise`, and `must_reintroduce`, and runtime jumps to that event. Boiler should bind Scene 1 to that start event.

14.5 Cold open rule

Do not bake the Phase 3 cold open into the cached runtime prompt.

The existing architecture uses `[OPENING_SCENE_INJECTION_POINT]`, with ChaosEngineService injecting the cold open once at live session start.

Boiler may reference the cold open scene, but it must not duplicate the cold open prose.

Recommended field:

```
{
  "cold_open_binding": {
    "uses_phase_3_cold_open": true,
    "scene_id": "S1_SCENE_01",
    "start_event_id": 123
  }
}
```

---

15. Runtime Prompt Additions

Add a new section to `runtime-narrator-template.blade.php`:

```
SECTION X: PLAYABLE SESSION CUT

If PLAYABLE_SESSION_CUT is present, this is your active structure.
Do not walk through raw events.
Do not narrate cut events.
Do not summarize the source event range.
Stage only the current playable scene.

CURRENT PLAYABLE SCENE:
{{current_playable_scene_card}}

SESSION SPINE:
{{compact_session_spine}}

RUNTIME PACING CONTRACT:
{{runtime_pacing_contract}}

SCENE EXIT CONDITION:
{{exit_condition}}

When the exit condition is met, stop cleanly and allow the engine to advance.
```

15.1 Runtime hard rules in Boiler mode

```
Resolve the player’s latest action first.
Advance at most one meaningful scene beat.
Stay inside the current scene card.
Use the scene’s word budget.
Ask for input early.
Do not cover future scenes.
Do not narrate montage/cut material unless the current scene card explicitly
tells you to.
End on live pressure, not explanation.
```

The old runtime already had an anti-autopilot principle: respond to the player first, advance only a small playable slice, and stop to hand control back. Boiler mode should preserve that but give it better scene structure.

---

16. ChaosEngineService Changes

16.1 Read playable session cut

When building runtime context:

```
$sessionAdaptation = ...
$boiler = $sessionAdaptation->playable_session_cut ?? null;
```

If Boiler exists and completed:

```
$currentScene = PlayableSceneResolver::resolve($game, $sessionAdaptation);
```

Inject:

```
current scene card
scene turn count
compact session spine
runtime pacing contract
scene-specific choices
scene-specific consequences
scene-specific world hooks
```

16.2 Scene advancement

Add resolver logic:

```
if ($currentSceneExitConditionMet) {
    $nextScene = $resolver->nextScene($currentScene);

    $game->current_playable_scene_id = $nextScene['scene_id'];
    $game->current_playable_scene_index = $nextScene['scene_order'];
    $game->playable_scene_turn_count = 0;
    $game->current_event_id = $nextScene['anchor_event_id'];

    $game->save();
}
```

16.3 Exit condition detection

V1 should keep this simple.

The narrator can emit a structured metadata field internally, or the backend can infer from selected choice ID / scene completion.

Recommended V1:

```
{
  "scene_progress": {
    "scene_id": "S1_SCENE_01",
    "exit_condition_met": true,
    "exit_reason": "Player defined posture toward RENDAN shard.",
    "next_scene_hint": "S1_SCENE_02"
  }
}
```

If current response format does not support metadata yet, store scene advancement after recognized choice resolution.

Do not overbuild this in V1.

16.4 Turn count

Each player → narrator exchange increments:

```
playable_scene_turn_count += 1
```

If count exceeds scene budget:

```
Runtime receives stronger gravity instruction:
Resolve or compress toward exit condition now.
```

---

17. Choice / Consequence Integration

Boiler should not replace ChoiceDesign or ConsequenceMapping.

It should reference them.

In each scene card:

```
"choice_links": [
  {
    "choice_id": "S1_C1",
    "choice_type": "branching",
    "dimension": "compliance_vs_defiance",
    "labels_available": ["A", "B", "C"]
  }
]
```

Consequence hooks:

```
"consequence_hooks": [
  {
    "choice_id": "S1_C1",
    "immediate_surface": "Ilya changes tone.",
    "delayed_surface": "Amora frames Nora differently in Session 2.",
    "world_state_keys": ["nora_compliance_status"]
  }
]
```

Freeform input should still classify into:

```
expressive
branch-aligned
emergent
unsupported
```

Current runtime already has this classification model. Boiler adds better scene affordances so freeform has cleaner rails.

---

18. EditorialVerification Updates

EditorialVerification should now verify Boiler.

Add checks:

18.1 Coverage check

Every session event is assigned exactly one role.

Fail if:

```
event missing from event_role_map
event assigned multiple roles
major turning point is CUT without reintroduction route
```

18.2 Canon check

Fail if Boiler:

```
contradicts source
changes required canon outcome
moves a future reveal too early
creates impossible world logic
removes required object/NPC continuity
```

18.3 Playability check

Fail if:

```
first input is too late
too many PLAY scenes in a row without decision/reaction
too many events are walked through
scene has no live pressure
scene has no player affordance
scene lacks exit condition
```

18.4 Pacing check

Flag if:

```
default word budget > 220
first input max > 220
scene turn budget missing
more than 4 turns before gravity
```

18.5 Replayability check

Fail or warn if:

```
branch choices do not affect later scene surfaces
session end has no identity reflection
objects/NPCs do not remember player posture
```

18.6 Voice surface check

Warn if:

```
cool extracts missing
object anchors missing
author-specific diction not represented
Boiler summaries feel generic
```

Anima is a useful canary because its audit already shows strong latent choice architecture, bounded agency, emotional range, and natural replayability hooks through networks, tethers, relays, decodes, and authentic-versus-simulated connection.

---

19. RuntimeNarratorAssembly Changes

19.1 Builder requirements

In Boiler-enabled mode, RuntimeNarratorAssembly should require:

```
voice_anchor
anchor_card
runtime_self_check
playable_session_cut
```

But keep compatibility:

```
If story.boiler_enabled = false:
    old requirements only.
If story.boiler_enabled = true:
    hard fail without playable_session_cut.
```

19.2 Prompt size impact

Boiler should reduce prompt size pressure.

Instead of rendering large session event context, render:

```
compact session spine
current scene card
nearby scene bridge
active choices/consequences
relevant world state
```

Continue respecting the existing V2.3 65k quality/control cap and never trim Anchor Card, Runtime Self-Check, Voice Anchor below minimum, or the opening injection slot.

---

20. Boiler V1 JSON Example

```
{
  "boiler_version": "1.0",
  "session_number": 1,
  "target_playtime_minutes": 25,
  "source_format": "SCREENPLAY",
  "playable_promise": "You are Nora inside clinical grief-tech pressure,
deciding whether to remain a tool of Amora or break the clean channel.",
  "session_spine": [
    "Wake inside another person's cleared grief.",
    "Ilya introduces RENDAN pressure.",
    "Nora defines her first posture toward compliance.",
    "The dead become personal through Eli and the lens.",
    "The Mirror reveals that comfort is not mercy.",
    "Nora leaves watched, changed, and carrying the wrong object."
  ],
  "event_role_map": [
    {
      "event_id": 1001,
      "event_position": 1,
      "role": "PLAY",
      "included_in_scene_id": "S1_SCENE_01",
      "why": "Immediate embodied pressure and first object/NPC interaction."
    },
    {
      "event_id": 1002,
      "event_position": 2,
      "role": "CHOICE",
      "included_in_scene_id": "S1_SCENE_01",
      "why": "First compliance-versus-defiance posture."
    }
```

```
  ],
  "playable_scenes": [
    {
      "scene_id": "S1_SCENE_01",
      "scene_order": 1,
      "scene_type": "cold_open_play",
      "source_event_ids": [1001, 1002],
      "anchor_event_id": 1001,
      "title": "The RENDAN Shard",
      "duration_budget": {
        "target_turns": "2-3",
        "max_turns_before_gravity": 3,
        "first_input_max_words": 180,
        "default_response_words": "80-180",
        "setpiece_response_words": "220-320"
      },
      "opening_image":
"White light. Chrome ribs. A patient reaching for someone the loop has erased.",
      "live_pressure": "Ilya places the RENDAN shard between Nora and the
exit.",
      "player_affordances": [
        "take the shard",
        "refuse the shard",
        "ask what RENDAN means",
        "inspect the patient",
        "pocket it without agreeing"
      ],
      "interaction_type": "open_input",
      "choice_links": [
        {
          "choice_id": "S1_C1",
          "dimension": "compliance_vs_defiance"
        }
      ],
      "canon_must_keep": [
        "Nora clears grief loops professionally.",
        "Ilya wants clean compliance.",
        "RENDAN matters.",
        "Nora's first posture toward Amora is established here."
      ],
      "cool_extracts": [
        "GRIEF LOOP / CLEARED",
        "RENDAN",
        "CLEAR AND REPLACE? Y/N"
      ],
      "object_anchors": [
        "RENDAN shard",
        "pod glass",
```

```
        "patient monitor"
      ],
      "forbidden_expansions": [
        "Do not explain the whole SoulSync system here.",
        "Do not narrate future Eli/Mirror reveals.",
        "Do not make Ilya openly villainous."
      ],
      "exit_condition":
"The player has defined Nora's first posture toward Ilya and the RENDAN shard.",
      "next_scene_bridge": "Move fast toward the private cost of refusing clean
grief."
    }
  ],
  "montage_bridges": [
    {
      "bridge_id": "S1_BRIDGE_01",
      "from_scene_id": "S1_SCENE_01",
      "to_scene_id": "S1_SCENE_02",
      "source_event_ids": [1003],
      "compression_style": "hard cinematic cut",
      "must_include": [
        "lab corridor",
        "clinical cold",
        "Nora carrying or refusing the shard state"
      ]
    }
  ],
  "cut_material": [
    {
      "source_event_id": 1004,
      "reason_cut": "Exposition slows the opening.",
      "reintroduction_route": "Bring back through HUD text and Ilya's later
phrasing."
    }
  ],
  "runtime_pacing_contract": {
    "first_input_max_words": 180,
    "default_response_words": "80-180",
    "major_reveal_words": "220-320",
    "max_exploratory_beats_before_gravity": 3,
    "do_not_walk_through_cut_events": true
  }
}
```

21. Implementation Sprints

Sprint 0 — Product Spec Lock

Deliverables:

```
Boiler schema v1
Boiler event roles
Boiler runtime mode rules
Boiler QA checklist
Feature flag decision
```

Engineering output:

```
docs/boiler-project.md
docs/boiler-schema-v1.json
```

Decision required:

```
Will Boiler be story-level opt-in, environment feature flag, or default for new
adaptations?
```

Recommended:

```
config feature flag + story/adaptation-level opt-in
```

Sprint 1 — Storage and Schema

Code changes:

```
Add migration for session_adaptations.playable_session_cut fields.
Add optional runtime scene state storage.
Create BoilerPlayableSessionCutSchema.
Create BoilerPlayableSessionCutValidator.
```

Acceptance criteria:

```
Migration runs.
Existing sessions unaffected.
Validator rejects missing scene IDs, missing event role map, missing exit
conditions.
```

Sprint 2 — Boiler Agent and Prompt

Code changes:

```
Create BoilerGenerationAgent.
Create BoilerGenerationJob.
Create BoilerPromptContextBuilder.
Create system-prompt.blade.php.
Create prompt.blade.php.
Add format-specific prompt branches for screenplay and novel.
```

Acceptance criteria:

```
Agent returns valid JSON.
Every source event has role.
Every PLAY/CHOICE scene has player affordances.
Every scene has word/turn budget.
Every scene has exit condition.
```

Sprint 3 — Pipeline Wiring

Code changes:

```
Insert BoilerGenerationJob after SessionCloseJob.
Update pipeline orchestration.
Ensure EditorialVerification receives playable_session_cut.
Add boiler_status transitions.
```

Status flow:

```
pending → generating → completed
pending → generating → failed
```

Acceptance criteria:

```
Pipeline can run with Boiler enabled.
Pipeline can run without Boiler enabled.
Failure stores boiler_error.
No partial invalid Boiler is marked completed.
```

Sprint 4 — Editorial Verification Upgrade

Code changes:

```
Patch EditorialVerification prompt to inspect Boiler.
Patch EditorialVerification schema if needed.
Add verification fields:
- boiler_coverage_pass
- boiler_canon_pass
- boiler_playability_pass
- boiler_pacing_pass
- boiler_replayability_pass
- boiler_voice_surface_pass
```

Acceptance criteria:

```
Verification fails/warns on missing event role coverage.
Verification flags first input too late.
Verification flags scene cards without affordances.
Verification flags cut major turning point without reintroduction.
```

Sprint 5 — Runtime Prompt Boiler Mode

Code changes:

```
Update runtime-narrator-template.blade.php.
Update RuntimeNarratorTemplateBuilder.
Add PLAYABLE_SESSION_CUT section.
Add CURRENT_PLAYABLE_SCENE section.
Prevent full event range rendering as primary material in Boiler mode.
```

```
Preserve voice anchor/card/self-check behavior.
Preserve [OPENING_SCENE_INJECTION_POINT].
```

Acceptance criteria:

```
Runtime prompt includes current scene card.
Runtime prompt includes session spine.
Runtime prompt includes pacing contract.
Runtime prompt does not duplicate cold_open.
Runtime prompt does not render full event range as active script in Boiler mode.
```

Sprint 6 — ChaosEngine Runtime Scene Cursor

Code changes:

```
Create PlayableSceneResolver.
Initialize current playable scene on game/session start.
Track scene turn count.
Advance scenes when exit condition is met.
Update current_event_id to next scene anchor_event_id.
Log completed scene IDs.
```

Acceptance criteria:

```
Game starts at Phase 3 start_event_id.
Current playable scene matches start_event_id.
Scene turn count increments.
Scene transition updates current_playable_scene_id and current_event_id
together.
Old runtime still works when Boiler is absent.
```

Sprint 7 — Choice and Freeform Integration

Code changes:

```
Map current scene choice_links into runtime choice presentation.
Ensure selected defining_choice_id still resolves normally.
```

```
Ensure freeform classification uses scene affordances.
Log branch-aligned/freeform actions against current scene.
```

Acceptance criteria:

```
Branch choices still update tracked_dimensions.
Freeform expressive actions stay local.
Freeform branch-aligned actions map to valid dimensions.
Unsupported actions fold back into current scene pressure.
```

Sprint 8 — Session End Card

Add build-time spec and runtime generation support for:

```
what the player changed
what the world remembers
alignment/path pressure
objects changed
NPC relationship shifts
next-session hook
```

Example:

```
Session 1 complete.

The world remembers:
- You refused Ilya’s clean channel.
- You kept the shard out of Amora’s hand.
- The patient saw you hesitate.

You are becoming:
Unfiltered.

Next:
The city starts answering through glass.
```

Acceptance criteria:

```
Session end card references actual logged choices.
No fake stats in V1.
```

```
No generic recap.
Card uses story-native language.
```

---

22. QA Plan

22.1 Canary stories

Start with:

```
Anima Machina Session 1
Anima Machina Session 5
Oz Session 1
Sherlock Session 1
```

Why Anima:

- Strong source voice.
- Strong object anchors.
- Clear branch dimensions.
- High risk of over-exposition because original IP has low recognizability. The audit specifically notes that Session 1 must onboard SoulSync, the Mirror, neural ports, grief loops, and Nora's stakes from scratch through action, not lore delivery.

22.2 QA metrics

Measure:

```
Words before first input.
Average words per runtime response.
Number of player inputs per 20 minutes.
Number of scenes completed.
Number of branch hooks surfaced.
Number of world reactions.
Prompt size before and after Boiler.
User-perceived pacing.
Canon violation count.
Voice drift count.
```

22.3 Manual QA questions

For each session, ask:

```
Did I get to act quickly?
Did the world react to me?
Did I feel inside the author’s world?
Did I understand enough without lore dump?
Did the scene move?
Did my choice create identity?
Would I replay this?
Would I continue to Session 2?
```

22.4 Failure signs

Boiler failed if:

```
It reads like a summary.
It preserves too many events as PLAY.
It cuts too much and loses stakes.
It makes choices feel random.
It creates a generic game loop.
It weakens author voice.
It removes canon anchors.
It lets runtime walk through events anyway.
```

---

23. Engineer Task List

Backend

```
1. Add migration for playable_session_cut and boiler status fields.
2. Add BoilerGenerationJob.
3. Add BoilerGenerationAgent.
4. Add Boiler schema and validator.
5. Add BoilerPromptContextBuilder.
6. Insert BoilerGenerationJob into pipeline after SessionCloseJob.
7. Pass Boiler output into EditorialVerification.
8. Add PlayableSceneResolver.
9. Add runtime scene state tracking.
10. Update ChaosEngineService to inject current playable scene in Boiler mode.
```

Prompt engineering

```
1. Write Boiler system prompt.
2. Write Boiler user prompt.
3. Add screenplay-specific compiler rules.
4. Add novel-specific compiler rules.
5. Patch EditorialVerification to verify Boiler.
6. Patch runtime narrator template with Boiler mode.
7. Ensure cold_open is not baked or duplicated.
8. Ensure runtime still uses Voice Anchor / Anchor Card / Runtime Self-Check.
```

QA / Daniel

```
1. Run migrations.
2. Run Boiler canary on Anima Session 1.
3. Inspect session_adaptations.playable_session_cut.
4. Verify every source event has role.
5. Verify first playable scene binds to start_event_id.
6. Run runtime prompt assembly.
7. Inspect prompt for duplicate cold_open.
8. Inspect prompt for full event over-rendering.
9. Play 10–15 turns.
10. Record pacing, agency, canon, and voice failures.
```

---

24. Suggested Artisan Commands to Add

Add command:

```
php artisan lorespinner:generate-boiler {story_id} {--session=} {--force}
```

Add validation command:

```
php artisan lorespinner:validate-boiler {story_id} {--session=}
```

Add inspection command:

```
php artisan lorespinner:inspect-boiler {story_id} {--session=}
```

Example expected use:

```
php artisan migrate

php artisan lorespinner:generate-boiler 25 --session=1 --force

php artisan lorespinner:validate-boiler 25 --session=1

php artisan lorespinner:inspect-boiler 25 --session=1
```

---

25. Risk Register

Risk 1: Boiler becomes summary

Mitigation:

```
Require player_affordances.
Require live_pressure.
Require object_anchors.
Require exit_condition.
Reject scene cards that only summarize.
```

Risk 2: Boiler cuts too much canon

Mitigation:

```
Require event_role_map coverage.
Require canon_must_keep.
Require reintroduction_route for cut major material.
EditorialVerification checks major turning points.
```

Risk 3: Runtime ignores Boiler

Mitigation:

```
Runtime prompt says current scene card is active structure.
Do not render full event text as primary active material.
Scene cursor keeps runtime anchored.
```

Risk 4: Voice weakens because full events are gone

Mitigation:

```
Keep Voice Profile.
Keep Anchor Card.
Keep Runtime Self-Check.
Include cool extracts and voice surface notes.
Include selected source texture, not full event bulk.
```

Risk 5: Engineering complexity too high

Mitigation:

```
V1 only tracks current scene ID and turn count.
V1 uses simple exit condition metadata.
V1 keeps old event progression fallback.
V1 is feature-flagged.
```

---

26. V1 Scope

Build only:

```
playable_session_cut generation
event role map
playable scene cards
runtime boiler section
scene cursor
basic scene advancement
editorial verification checks
Anima canary QA
```

Do not build yet:

```
automatic player stats dashboard
global player percentage comparison
social sharing cards
advanced scene graph branching
dynamic promotion of emergent branches
```

```
visual UI mode
multiplayer/co-op
```

---

27. Success Criteria

Boiler V1 is successful when:

```
1. Runtime no longer feels like it is walking through full events.
2. First player agency appears fast.
3. Default responses are shorter and more playable.
4. The player feels the world reacting to their posture.
5. Canon remains intact.
6. Voice remains author-specific.
7. Session prompt is smaller and cleaner.
8. QA players want to continue to the next session.
```

The most important subjective test:

> Does the player feel like they are driving the story forward inside the author's world?

If yes, Boiler is working.

---

28. Final Architecture Statement

Boiler is the missing third layer.

```
Canon Layer:
chapters → events
source truth

Adaptation Layer:
sessions → beats → choices → consequences
dramatic design

Boiler Layer:
playable scenes → interaction budgets → cut ledger → scene pressure
game feel

Runtime Layer:
current scene performance
```

```
player response
world reaction
```

Without Boiler, runtime has to convert script into play live.

With Boiler, runtime performs a designed playable cut.

That is the product leap.

Not shorter narration.

Playable storytelling.